In [1]:
```
# IBM HR ANALYTICS PROJECT
```

In [2]:
```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
```

In [3]:
```
df = pd.read_csv("D:\IBM HR data\WA_Fn-UseC_-HR-Employee-Attrition.csv")
df.head(5)
```

Out[3]:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Educat
0	41	Yes	Travel_Rarely	1102	Sales	1	
1	49	No	Travel_Frequently	279	Research & Development	8	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	
4	27	No	Travel_Rarely	591	Research & Development	2	

5 rows × 35 columns

In [4]:
```
print(df.shape)
df.info()
df.describe()
```

```
(1470, 35)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Age                       1470 non-null   int64
 1   Attrition                 1470 non-null   object
 2   BusinessTravel            1470 non-null   object
 3   DailyRate                 1470 non-null   int64
 4   Department                1470 non-null   object
 5   DistanceFromHome          1470 non-null   int64
 6   Education                 1470 non-null   int64
 7   EducationField            1470 non-null   object
 8   EmployeeCount             1470 non-null   int64
 9   EmployeeNumber            1470 non-null   int64
 10  EnvironmentSatisfaction   1470 non-null   int64
 11  Gender                    1470 non-null   object
 12  HourlyRate                1470 non-null   int64
 13  JobInvolvement            1470 non-null   int64
 14  JobLevel                  1470 non-null   int64
 15  JobRole                   1470 non-null   object
 16  JobSatisfaction           1470 non-null   int64
 17  MaritalStatus             1470 non-null   object
 18  MonthlyIncome             1470 non-null   int64
 19  MonthlyRate               1470 non-null   int64
 20  NumCompaniesWorked        1470 non-null   int64
 21  Over18                    1470 non-null   object
 22  OverTime                  1470 non-null   object
 23  PercentSalaryHike         1470 non-null   int64
 24  PerformanceRating         1470 non-null   int64
 25  RelationshipSatisfaction  1470 non-null   int64
 26  StandardHours             1470 non-null   int64
 27  StockOptionLevel          1470 non-null   int64
 28  TotalWorkingYears         1470 non-null   int64
 29  TrainingTimesLastYear     1470 non-null   int64
 30  WorkLifeBalance           1470 non-null   int64
 31  YearsAtCompany            1470 non-null   int64
 32  YearsInCurrentRole        1470 non-null   int64
 33  YearsSinceLastPromotion   1470 non-null   int64
 34  YearsWithCurrManager      1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
```

Out[4]:

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	Em
count	1470.000000	1470.000000	1470.000000	1470.000000	1470.0	
mean	36.923810	802.485714	9.192517	2.912925	1.0	
std	9.135373	403.509100	8.106864	1.024165	0.0	
min	18.000000	102.000000	1.000000	1.000000	1.0	
25%	30.000000	465.000000	2.000000	2.000000	1.0	
50%	36.000000	802.000000	7.000000	3.000000	1.0	
75%	43.000000	1157.000000	14.000000	4.000000	1.0	
max	60.000000	1499.000000	29.000000	5.000000	1.0	

8 rows × 26 columns

In [5]:
```
print(df.isnull().sum().sum())
print(df.duplicated().sum())
```

```
0
0
```

In [6]:
```
df = df.drop(columns=["EmployeeCount", "Over18", "StandardHours"])
```

In [7]:
```
df["Attrition_Flag"] = df["Attrition"].map({"Yes": 1, "No": 0})
```

In [8]:
```
df["AgeGroup"] = pd.cut(df["Age"], bins=[17, 25, 35, 45, 60],
                        labels=["18-25", "26-35", "36-45", "46-60"])
```

In [9]:
```
df["IncomeGroup"] = pd.qcut(df["MonthlyIncome"], 3,
                            labels=["Low", "Medium", "High"])
df["IncomeGroup"].value_counts()
```

Out[9]:
```
IncomeGroup
Low       490
Medium    490
High      490
Name: count, dtype: int64
```

In [10]:
```
df.to_csv("hr_clean.csv", index=False)
```

In [11]:
```
df.head(5)
```

Out[11]:

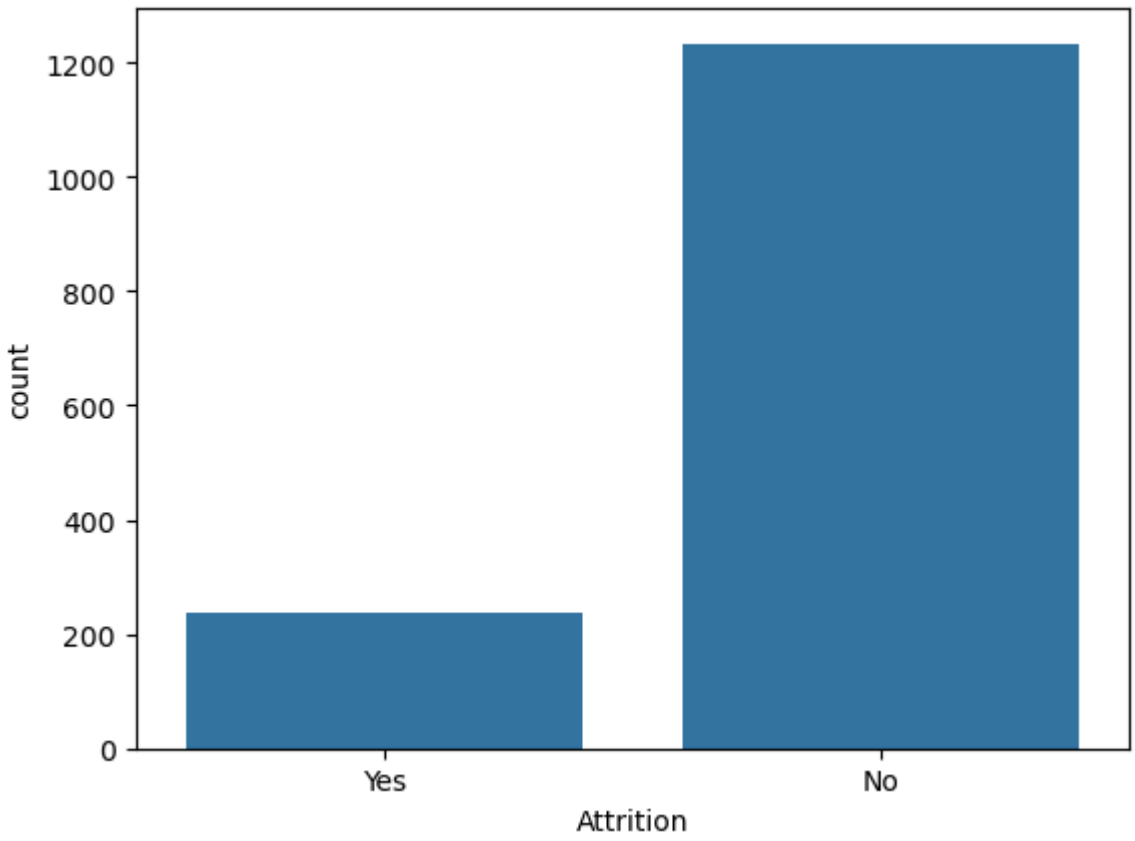
	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Educat
0	41	Yes	Travel_Rarely	1102	Sales	1	
1	49	No	Travel_Frequently	279	Research & Development	8	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	
4	27	No	Travel_Rarely	591	Research & Development	2	

5 rows × 35 columns

In [12]:
```
def attr_chart(col):
    result = df.groupby(col, observed=True)["Attrition_Flag"].mean() * 100
    print(result.round(1))
    result.plot(kind="bar")
    plt.ylabel("Attrition %")
    plt.show()
```

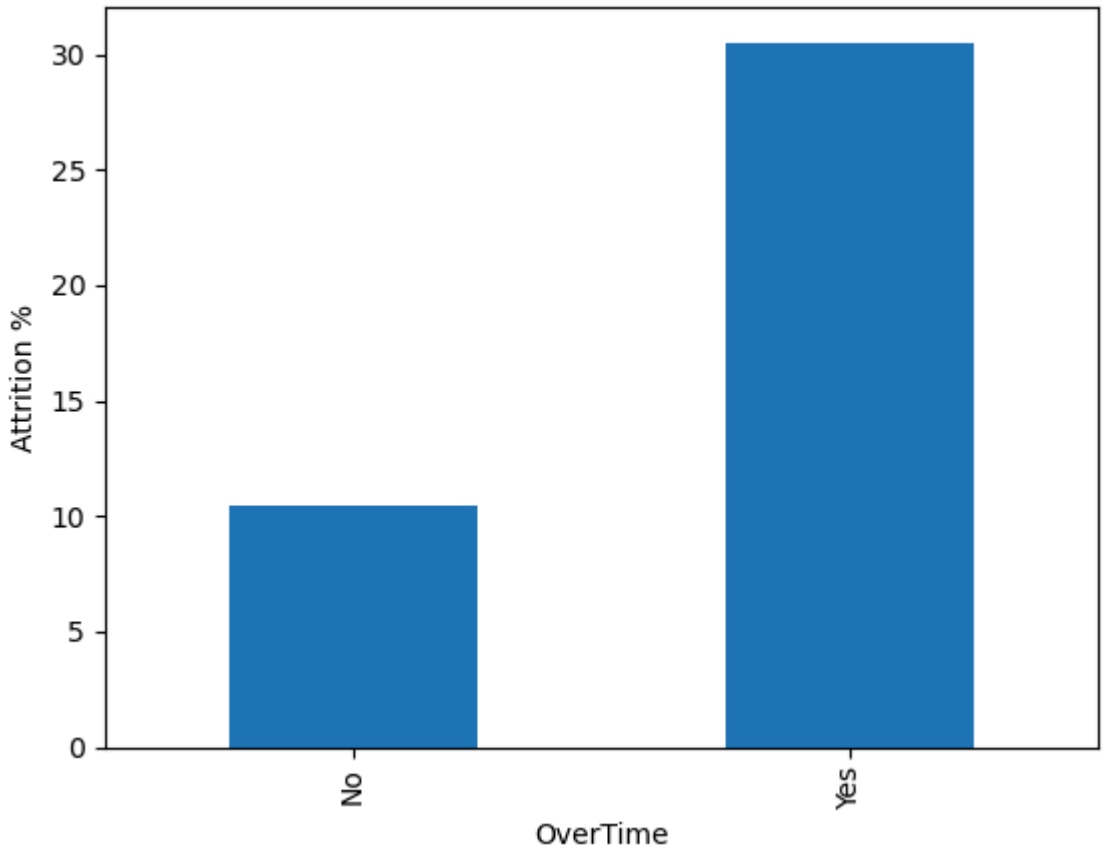
In [13]:
```
#Q1. What is the overall attrition rate?
print(df["Attrition_Flag"].mean() * 100)
sns.countplot(x="Attrition", data=df)
plt.show()
```

```
16.122448979591837
```



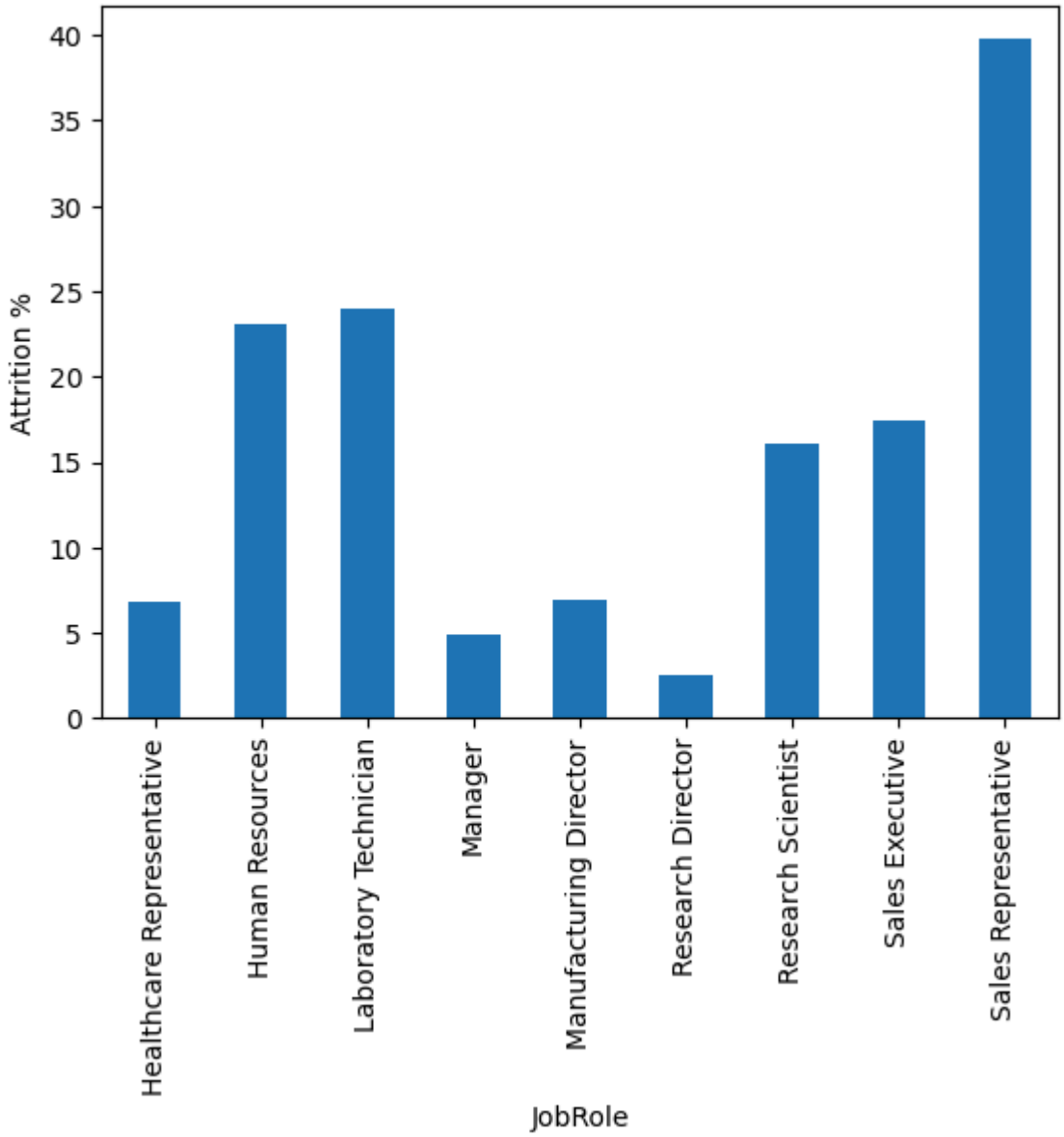
In [14]:
```
#Q2. Does overtime affect attrition?
attr_chart("OverTime")
```

```
OverTime
No     10.4
Yes    30.5
Name: Attrition_Flag, dtype: float64
```



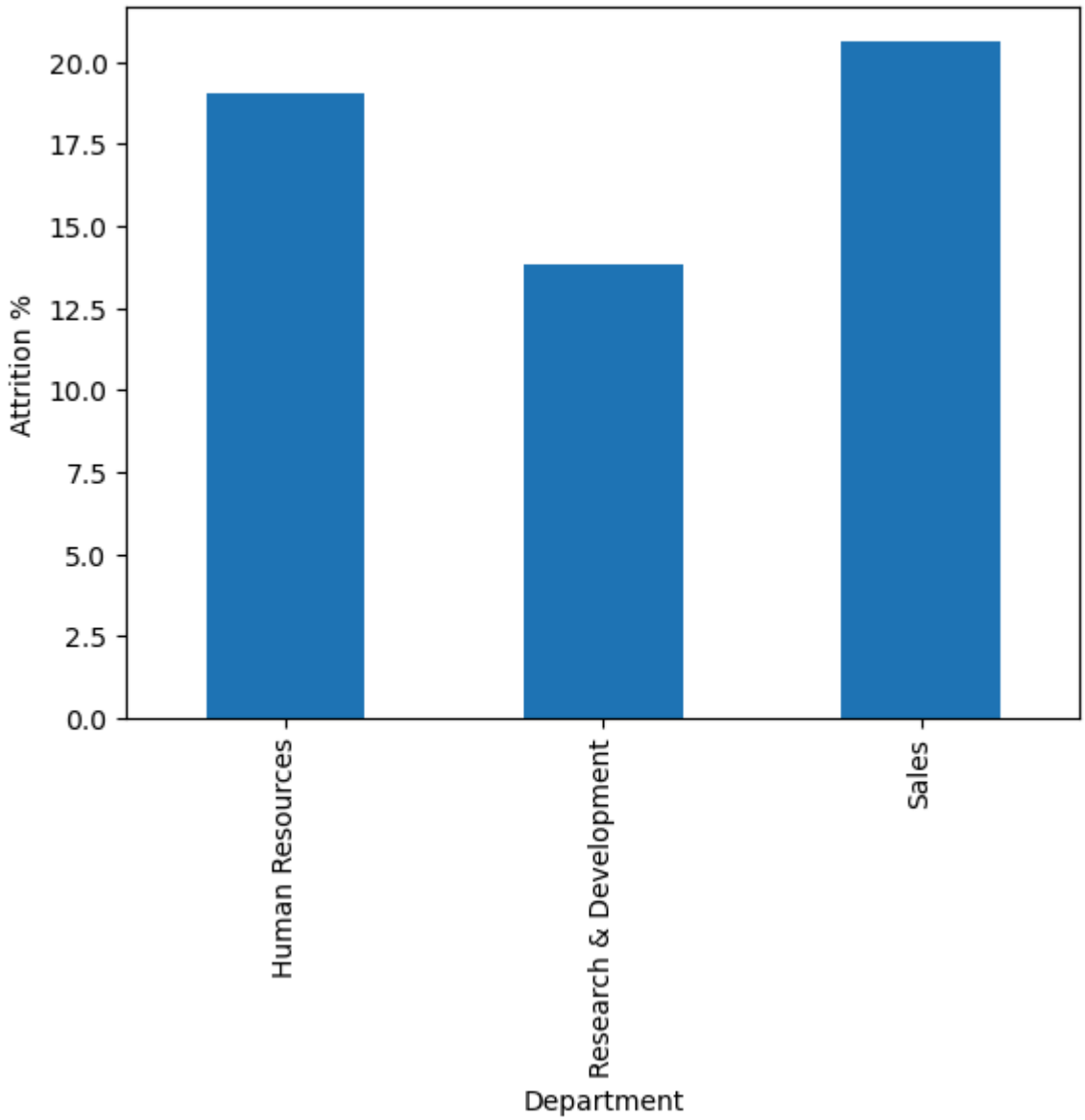
In [15]:
```
#Q3. Which job roles have the highest attrition?
attr_chart("JobRole")
```

```
JobRole
Healthcare Representative     6.9
Human Resources              23.1
Laboratory Technician        23.9
Manager                       4.9
Manufacturing Director        6.9
Research Director             2.5
Research Scientist           16.1
Sales Executive              17.5
Sales Representative         39.8
Name: Attrition_Flag, dtype: float64
```



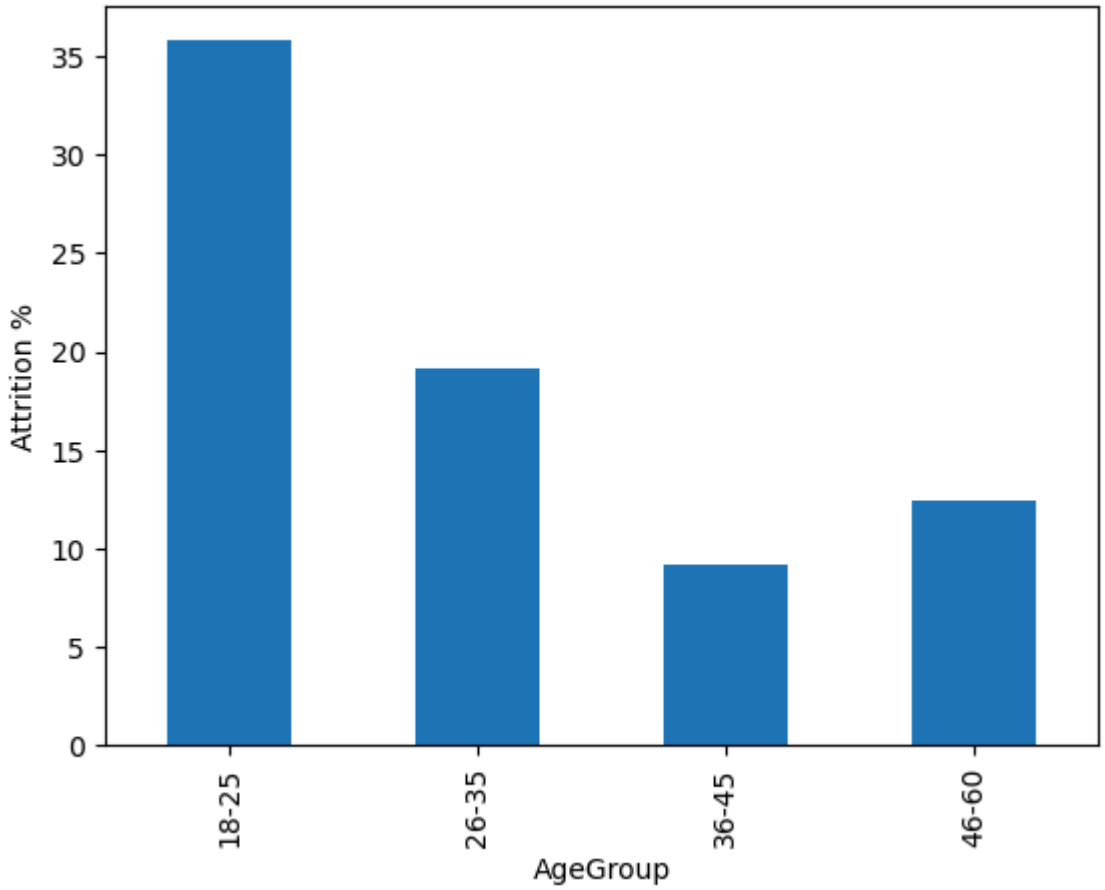
In [16]:
```
#Q4. Which department loses the most people?
attr_chart("Department")
```

```
Department
Human Resources           19.0
Research & Development    13.8
Sales                     20.6
Name: Attrition_Flag, dtype: float64
```



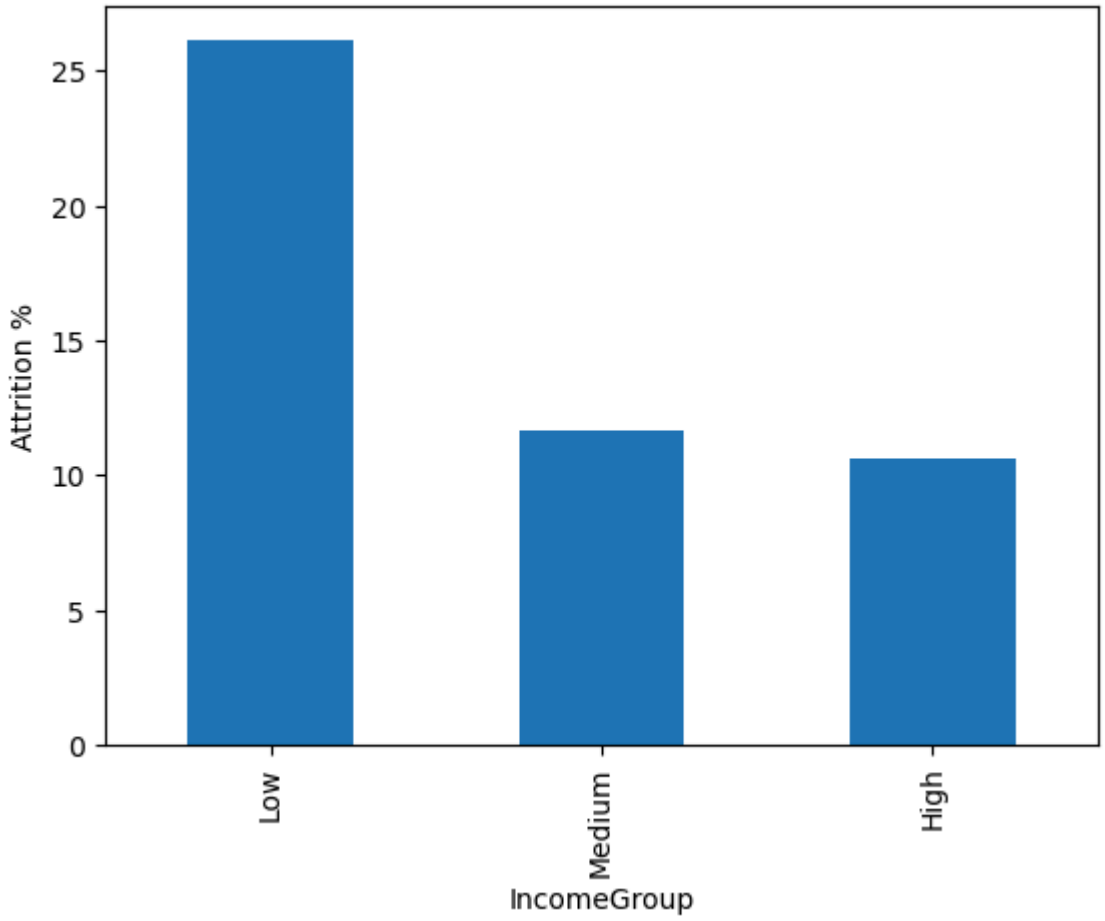
In [17]:
```
#Q5. Which age group leaves the most?
attr_chart("AgeGroup")
```

```
AgeGroup
18-25    35.8
26-35    19.1
36-45     9.2
46-60    12.5
Name: Attrition_Flag, dtype: float64
```

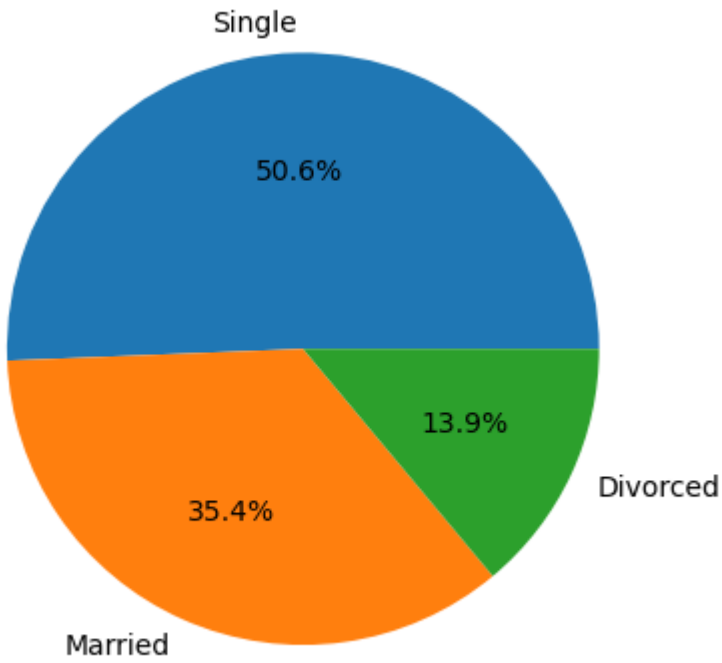


In [18]:
```
#Q6. Income
attr_chart("IncomeGroup")
```

```
IncomeGroup
Low       26.1
Medium    11.6
High      10.6
Name: Attrition_Flag, dtype: float64
```

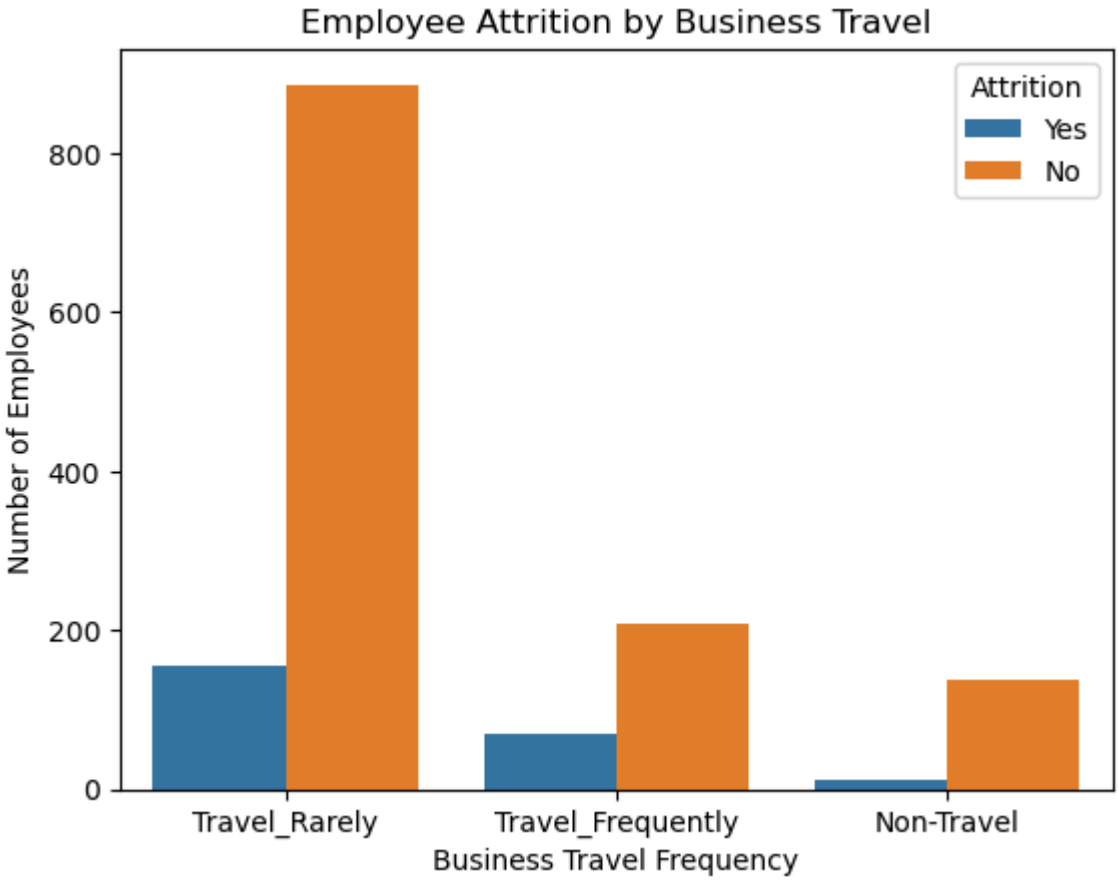


In [19]:
```
#Q7. Marital status
df[df["Attrition"] == "Yes"]["MaritalStatus"].value_counts().plot(kind="pie", au
plt.ylabel("")
plt.show()
```

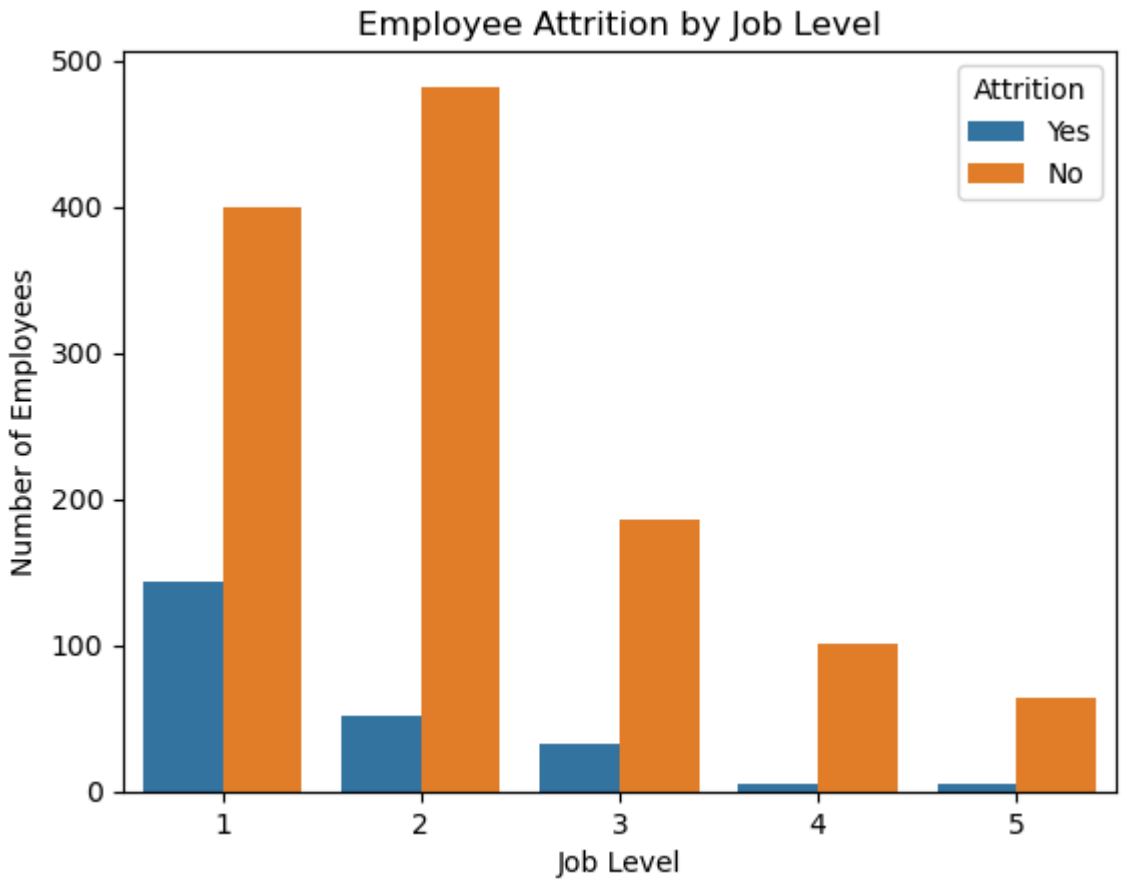


In [20]:
```
#Q8. Business travel
sns.countplot(x="BusinessTravel", hue="Attrition", data=df)
```

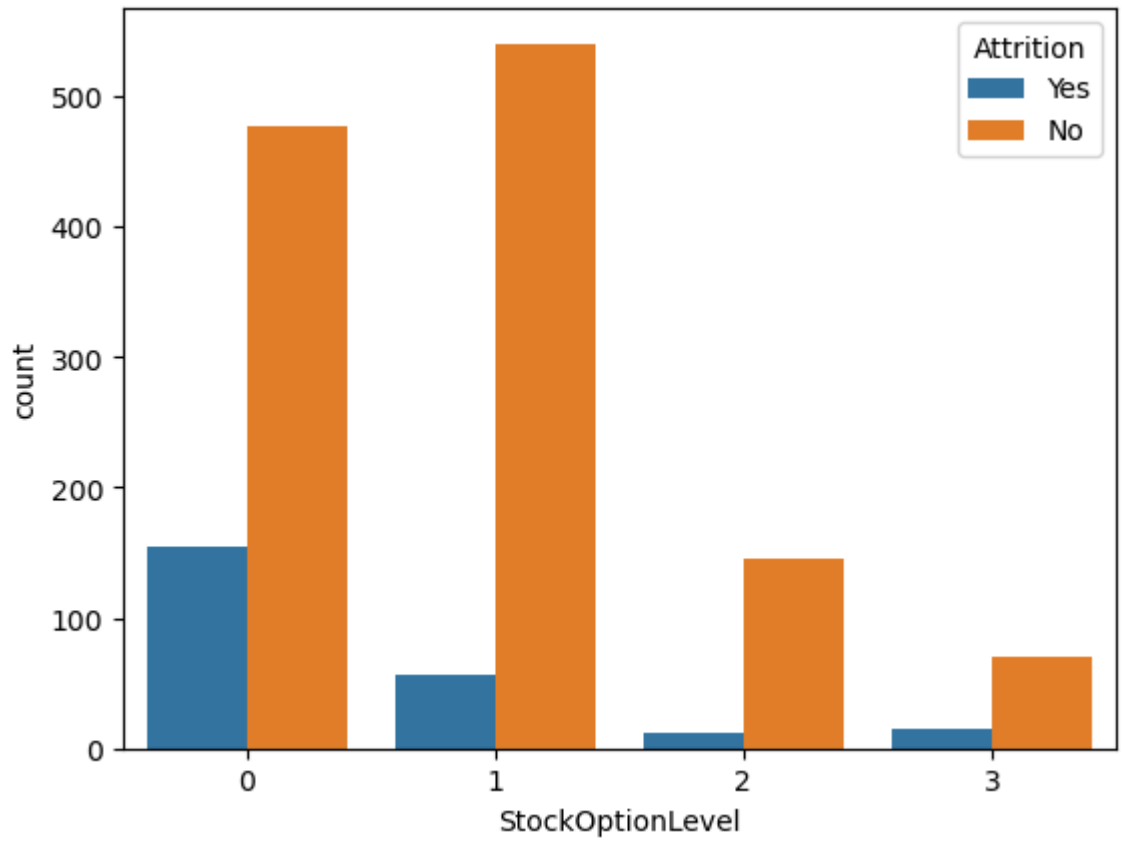
```
plt.title("Employee Attrition by Business Travel")
plt.xlabel("Business Travel Frequency")
plt.ylabel("Number of Employees")
plt.show()
```



In [21]:
```
#Q9. Job level
sns.countplot(x="JobLevel", hue="Attrition", data=df)
plt.title("Employee Attrition by Job Level")
plt.xlabel("Job Level")
plt.ylabel("Number of Employees")
plt.legend(title="Attrition")
plt.show()
```

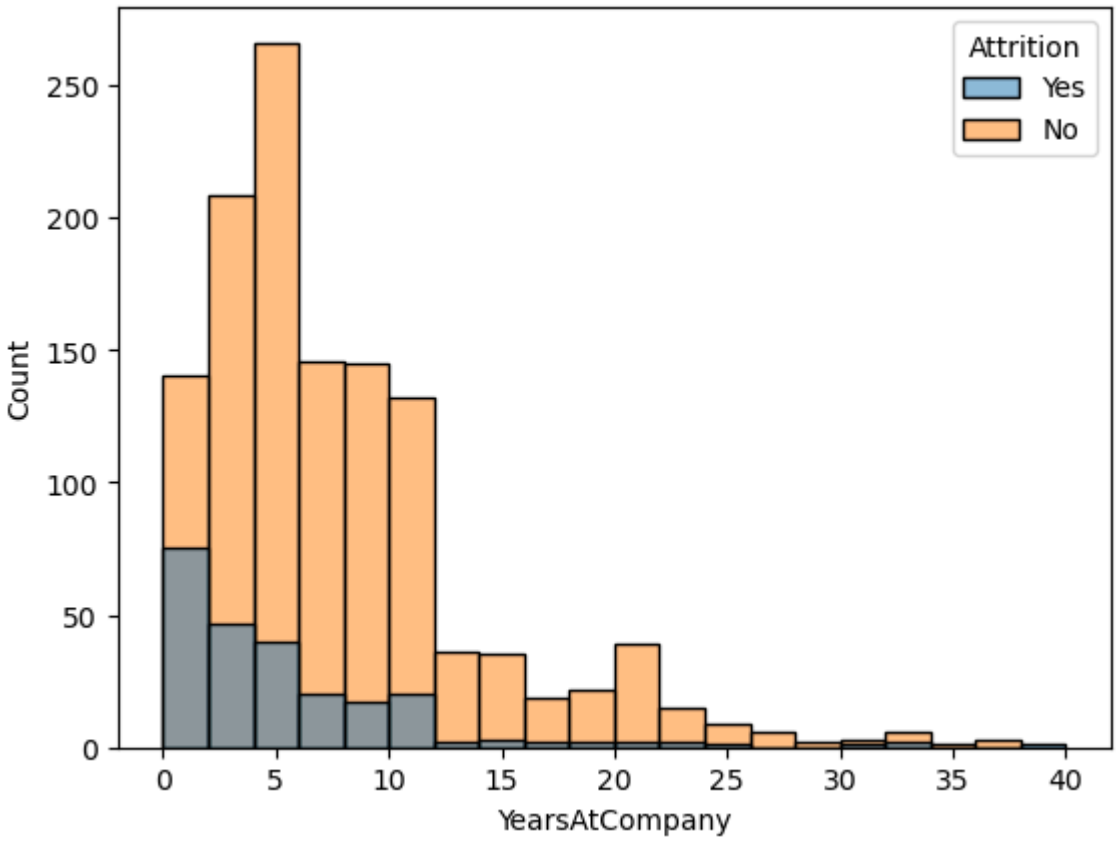


In [22]:
```
#Q10. Stock options
sns.countplot(x="StockOptionLevel", hue="Attrition", data=df)
plt.show()
```

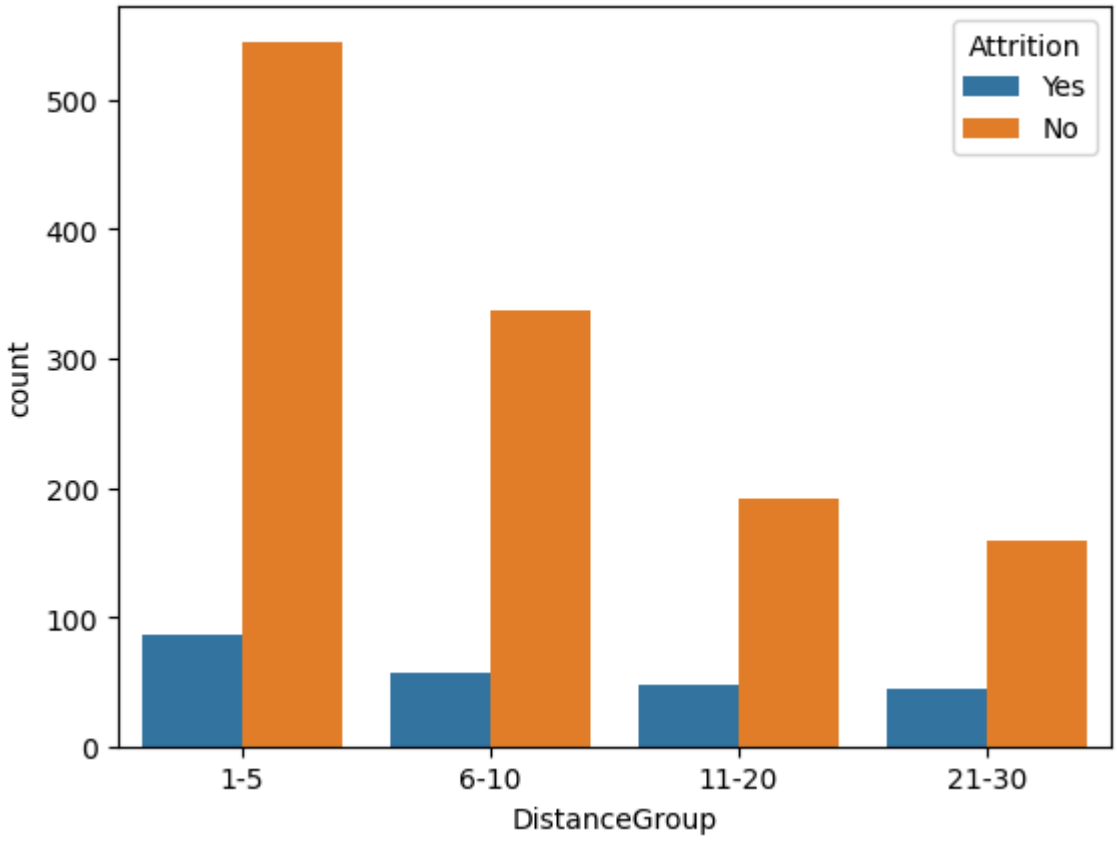


In [23]:
```
#Q11. Tenure
sns.histplot(data=df, x="YearsAtCompany", hue="Attrition", bins=20)
```

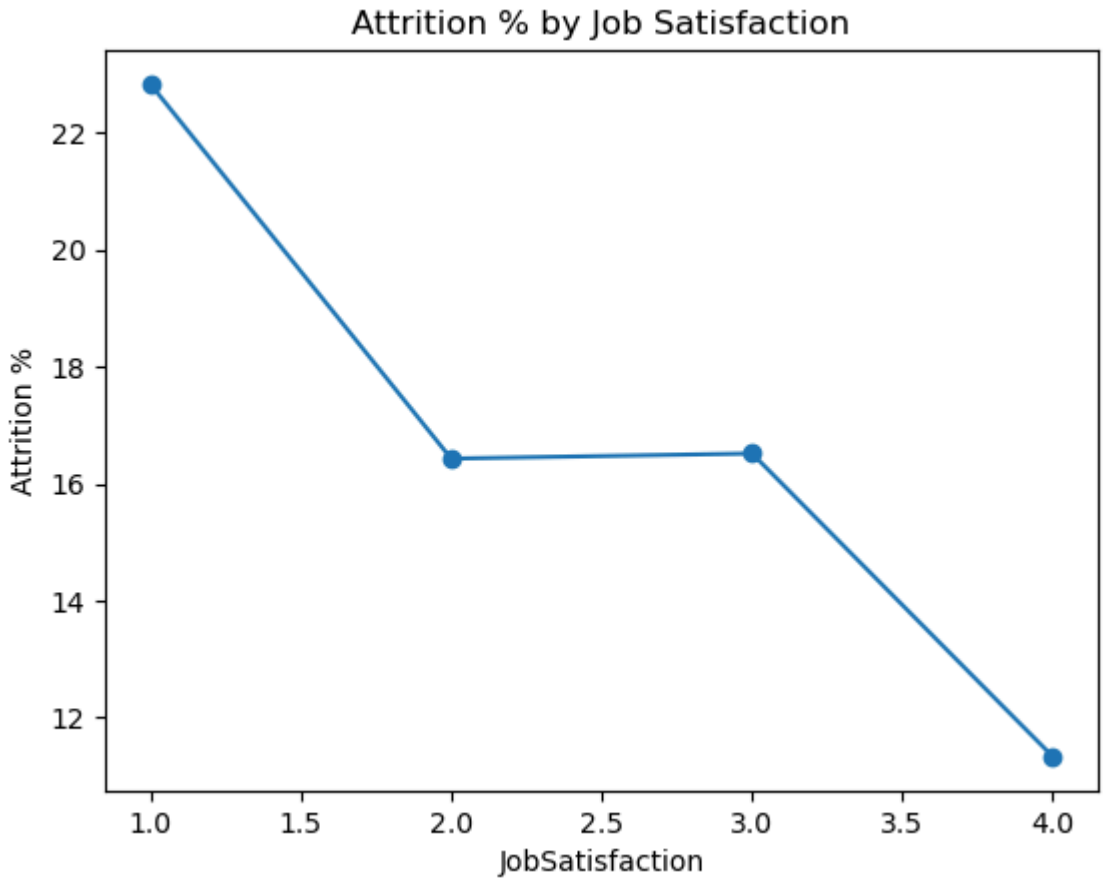
```
plt.show()
```



In [24]:
```
#Q12. Distance
df["DistanceGroup"] = pd.cut(df["DistanceFromHome"], bins=[0, 5, 10, 20, 30], la
sns.countplot(x="DistanceGroup", hue="Attrition", data=df)
plt.show()
```



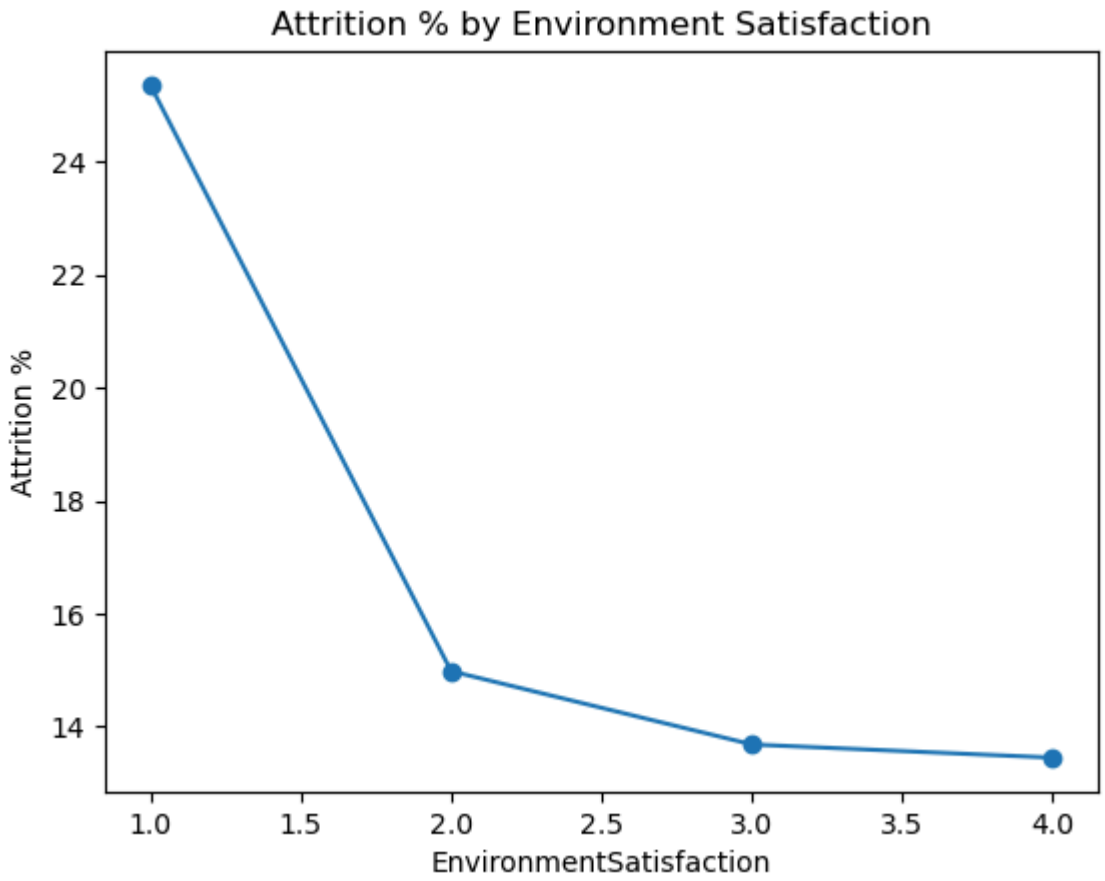
In [25]:
```
#Q13. Job satisfaction
rate = df.groupby("JobSatisfaction")["Attrition_Flag"].mean() * 100
rate.plot(kind="line", marker="o")
plt.title("Attrition % by Job Satisfaction")
plt.ylabel("Attrition %")
plt.show()
```



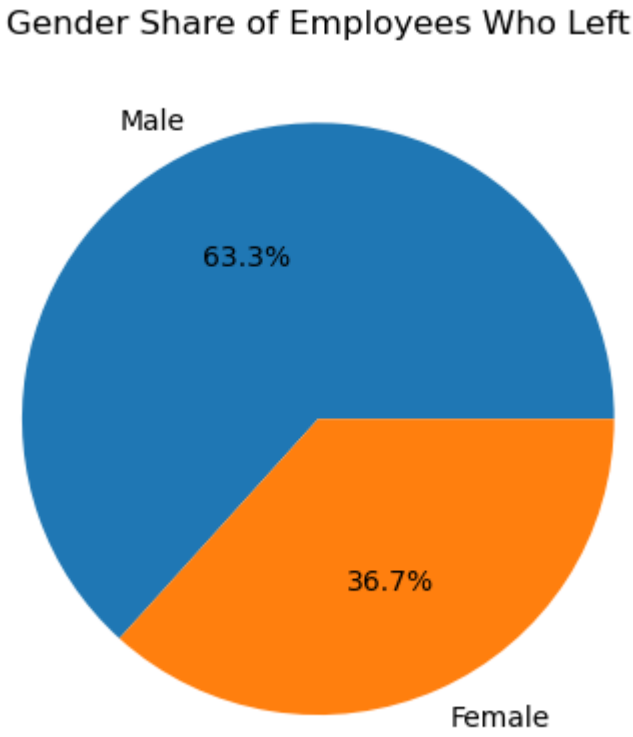
In [26]:
```
#Q14. Work-life balance
sns.countplot(x="WorkLifeBalance", data=df)
plt.title("Employees by Work-Life Balance")
plt.xlabel("Work-Life Balance Rating")
plt.ylabel("Number of Employees")
plt.show()
```



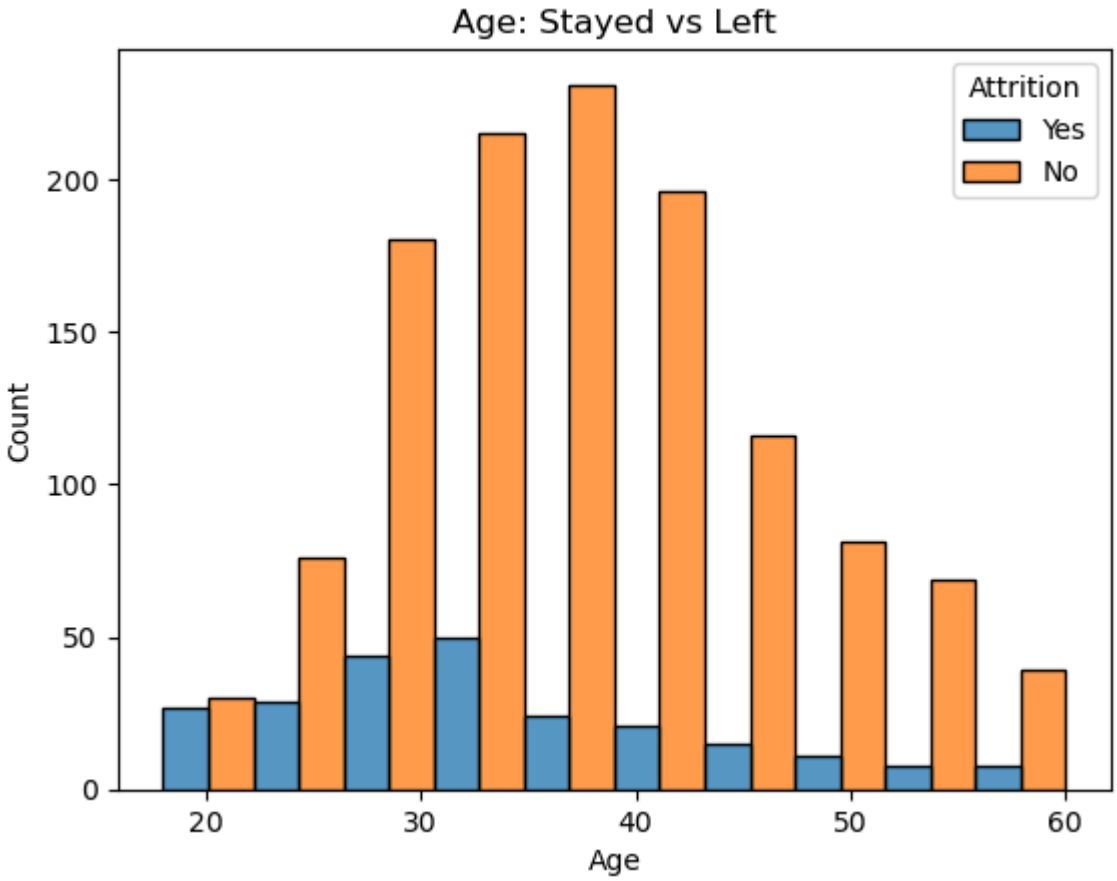
In [27]:
```
#Q15. Environment satisfaction
rate = df.groupby("EnvironmentSatisfaction")["Attrition_Flag"].mean() * 100
rate.plot(kind="line", marker="o")
plt.title("Attrition % by Environment Satisfaction")
plt.ylabel("Attrition %")
plt.show()
```



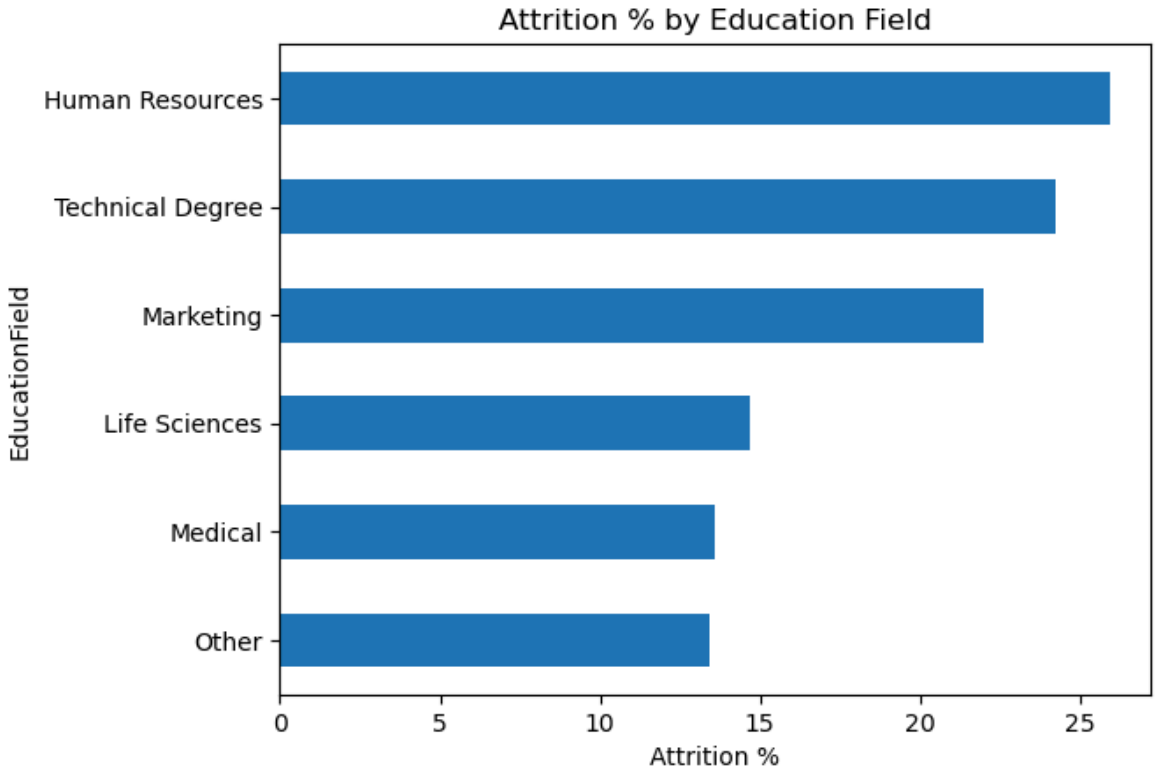
In [28]:
```
#Q16. Gender (share of leavers)
df[df["Attrition"] == "Yes"]["Gender"].value_counts().plot(kind="pie", autopct="
plt.title("Gender Share of Employees Who Left")
plt.ylabel("")
plt.show()
```



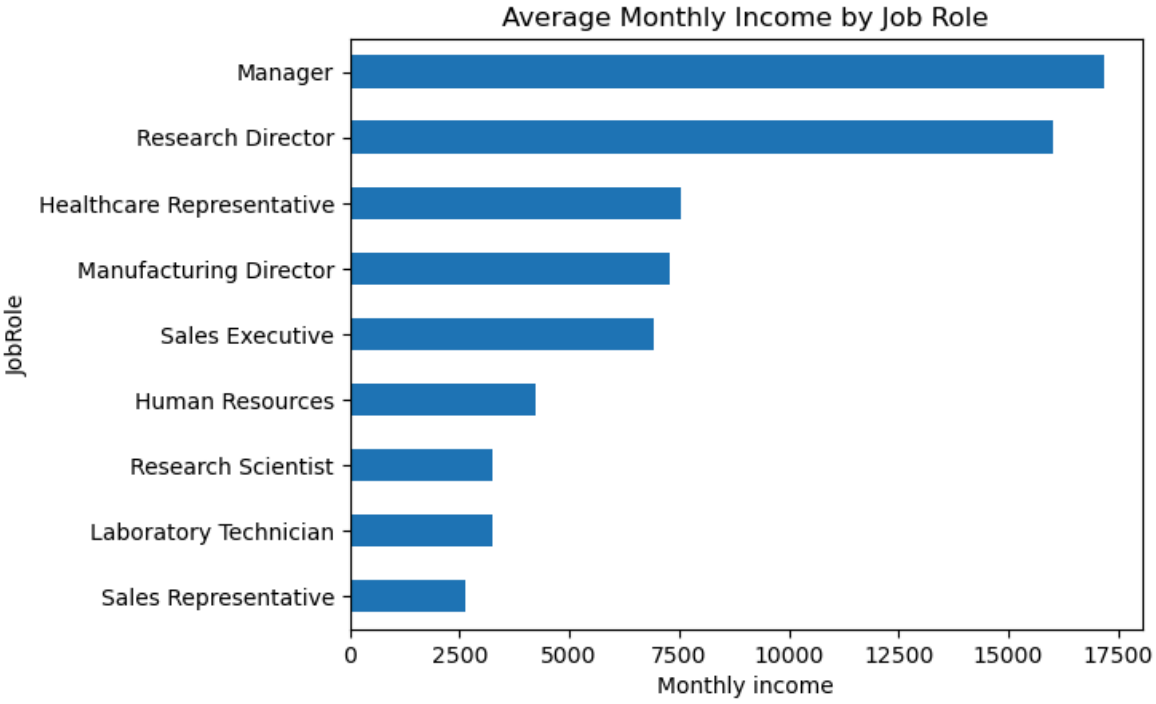
In [29]:
```
#Q17. Stayed vs Left
sns.histplot(data=df, x="Age", hue="Attrition", bins=10, multiple="dodge")
plt.title("Age: Stayed vs Left")
plt.show()
```



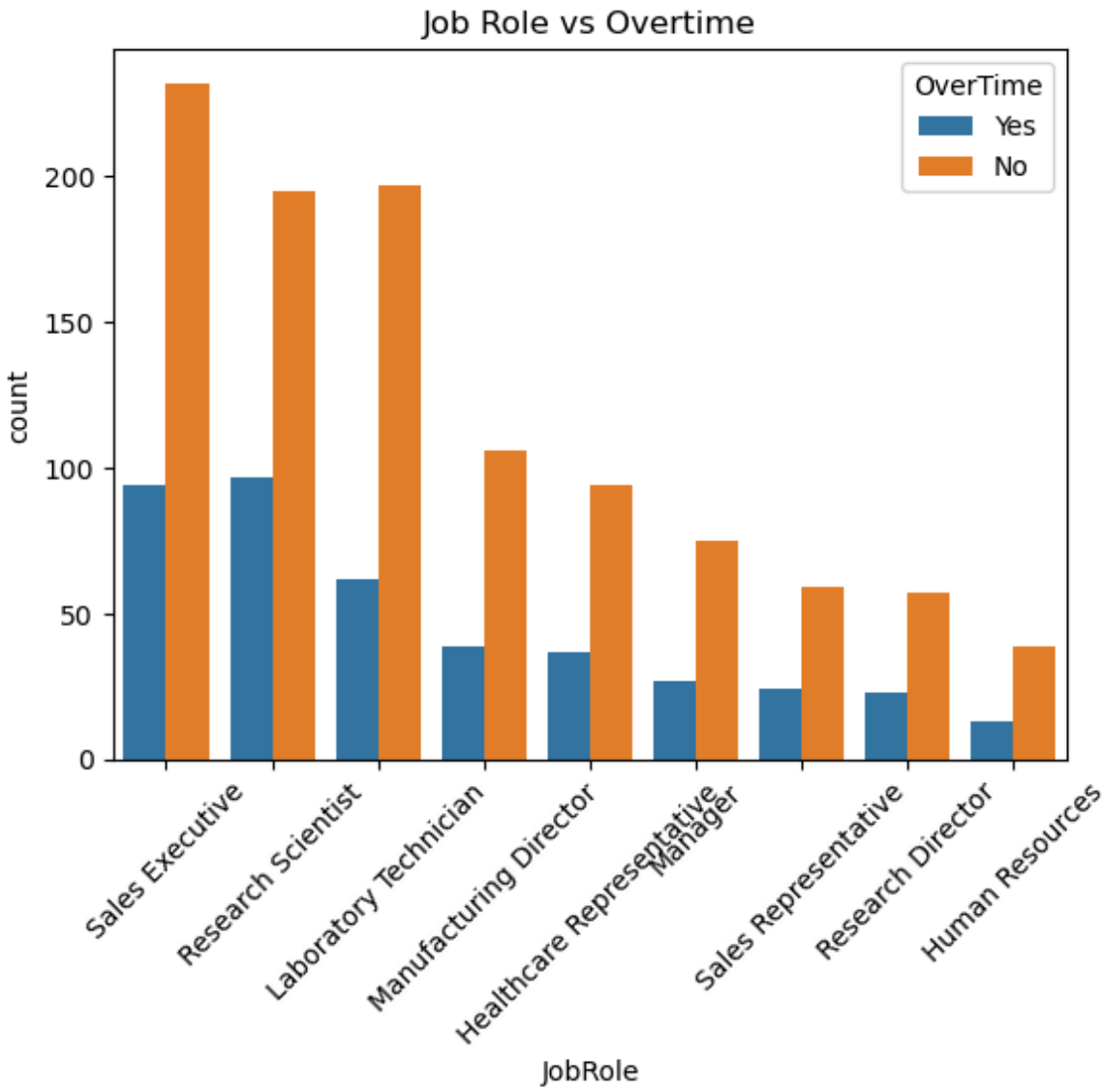
In [30]:
```
#Q18. Education field
rate = df.groupby("EducationField")["Attrition_Flag"].mean() * 100
rate.sort_values().plot(kind="barh")
plt.title("Attrition % by Education Field")
plt.xlabel("Attrition %")
plt.show()
```



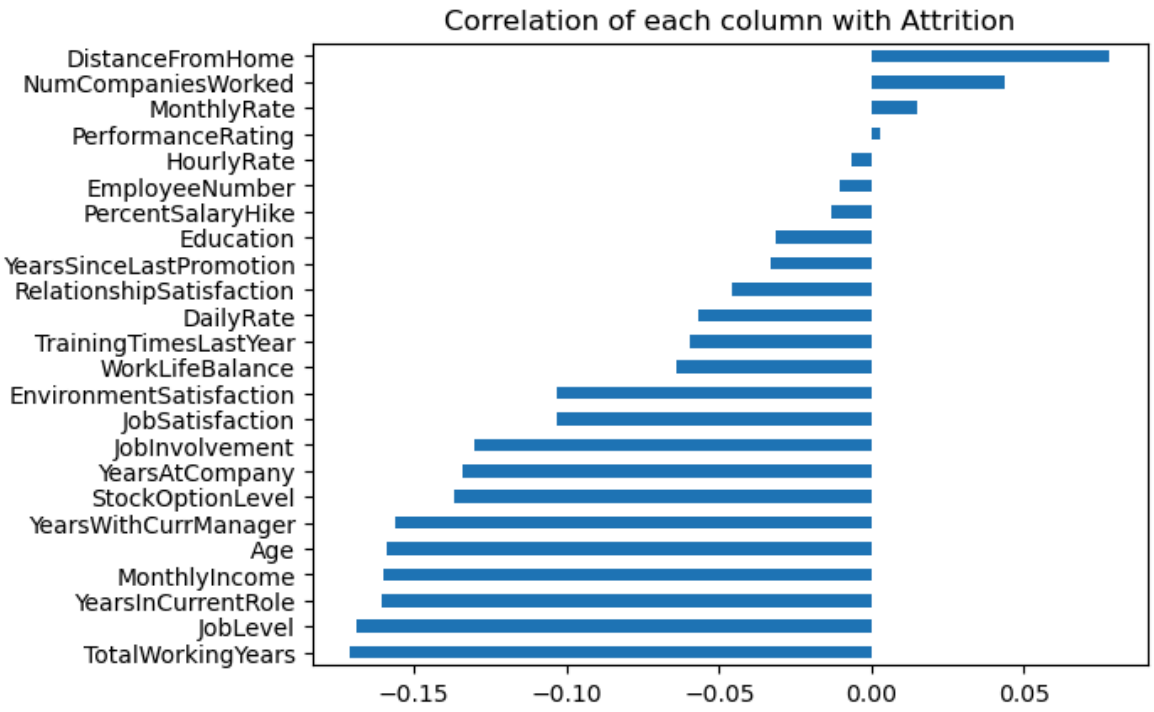
In [31]:
```
#Q19. Average income by role
avg = df.groupby("JobRole")["MonthlyIncome"].mean()
avg.sort_values().plot(kind="barh")
plt.title("Average Monthly Income by Job Role")
plt.xlabel("Monthly income")
plt.show()
```



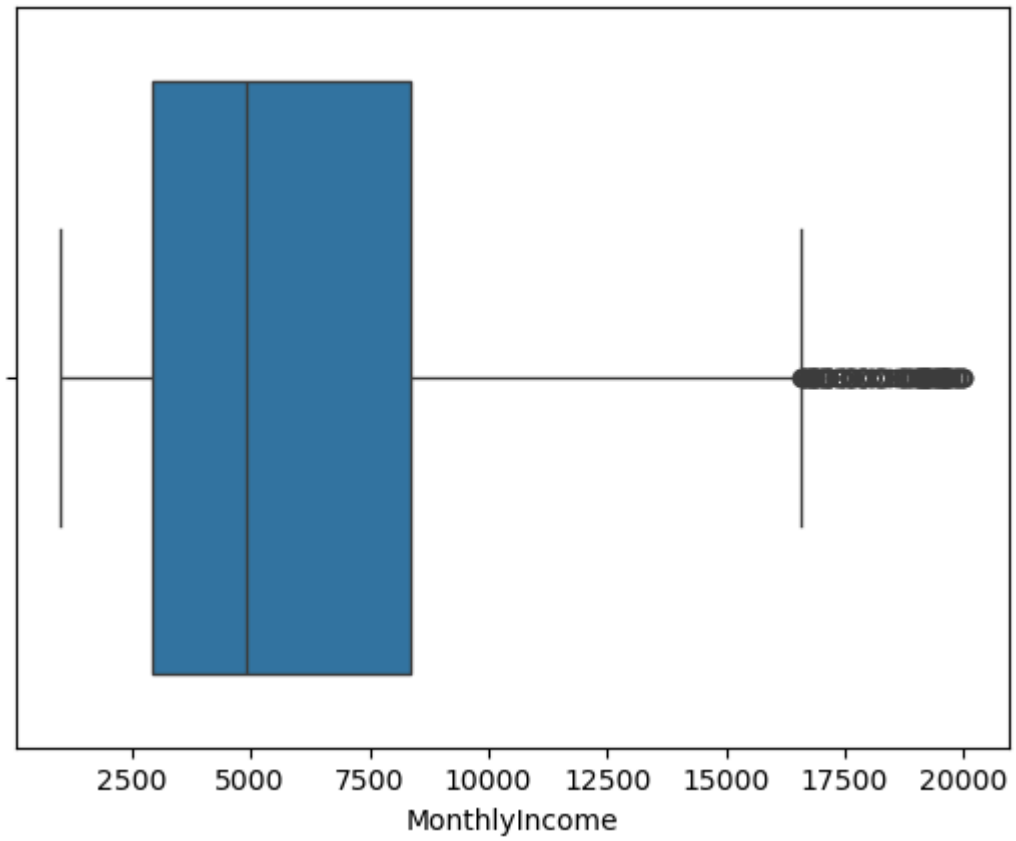
In [32]:
```
#Q22. Role × overtime
sns.countplot(data=df, x="JobRole", hue="OverTime")
plt.title("Job Role vs Overtime")
plt.xticks(rotation=45)
plt.show()
```



In [33]:
```
#Q23. Correlation with attrition
corr = df.select_dtypes("number").corr()["Attrition_Flag"].drop("Attrition_Flag"
corr.sort_values().plot(kind="barh")
plt.title("Correlation of each column with Attrition")
plt.show()
```



In [34]:
```
# Checking outliers
sns.boxplot(x=df["MonthlyIncome"])
plt.show()
```



In [35]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, recall_score, precision_score
```

In [36]:
```
y = df["Attrition_Flag"]

X = df.drop(columns=["Attrition", "Attrition_Flag", "EmployeeNumber",
                     "AgeGroup", "IncomeGroup", "TenureGroup", "DistanceGroup"],
            errors="ignore")
X = pd.get_dummies(X, drop_first=True)
```

In [37]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

In [38]:
```
#Prediction 1: Logistic Regression
model1 = LogisticRegression(max_iter=1000, class_weight="balanced")
model1.fit(X_train_s, y_train)
pred1 = model1.predict(X_test_s)

print("Accuracy:", accuracy_score(y_test, pred1))
print("Recall:", recall_score(y_test, pred1))
print("Precision:", precision_score(y_test, pred1))
```

```
Accuracy: 0.7517006802721088
Recall: 0.6170212765957447
Precision: 0.34523809523809523
```

In [39]:
```
#Prediction 2: Decision Tree
model2 = DecisionTreeClassifier(max_depth=4, class_weight="balanced", random_sta
model2.fit(X_train, y_train)
pred2 = model2.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred2))
print("Recall:", recall_score(y_test, pred2))
print("Precision:", precision_score(y_test, pred2))
```

```
Accuracy: 0.717687074829932
Recall: 0.48936170212765956
Precision: 0.2804878048780488
```

In [40]:
```
#Prediction 3: Random Forest
model3 = RandomForestClassifier(n_estimators=100, max_depth=8,
                                class_weight="balanced", random_state=42)
model3.fit(X_train, y_train)
pred3 = model3.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred3))
print("Recall:", recall_score(y_test, pred3))
print("Precision:", precision_score(y_test, pred3))
```

```
Accuracy: 0.826530612244898
Recall: 0.14893617021276595
Precision: 0.3888888888888889
```

In [41]:
```
#Step 4: Compare all three
results = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree", "Random Forest"],
    "Accuracy": [accuracy_score(y_test, pred1), accuracy_score(y_test, pred2), a
    "Recall": [recall_score(y_test, pred1), recall_score(y_test, pred2), recall_
    "Precision": [precision_score(y_test, pred1), precision_score(y_test, pred2)
}).round(2)
results
```

Out[41]:

	Model	Accuracy	Recall	Precision
0	Logistic Regression	0.75	0.62	0.35
1	Decision Tree	0.72	0.49	0.28
2	Random Forest	0.83	0.15	0.39

In [43]:
```
pred = pd.DataFrame({
    "EmployeeNumber": df["EmployeeNumber"],
    "Leave_Probability": model1.predict_proba(scaler.transform(X))[:, 1],
    "Pred_LogReg": model1.predict(scaler.transform(X)),
    "Pred_DecisionTree": model2.predict(X),
    "Pred_RandomForest": model3.predict(X),
})

pred["Risk_Level"] = pd.cut(pred["Leave_Probability"], bins=[-0.01, 0.3, 0.6, 1.
                            labels=["Low", "Medium", "High"]).astype(str)

for col in ["Pred_LogReg", "Pred_DecisionTree", "Pred_RandomForest"]:
    pred[col] = pred[col].map({1: "Yes", 0: "No"})

pred.to_sql("hr_predictions", engine, if_exists="replace", index=False)
pred["Risk_Level"].value_counts()
```

Out[43]:
```
Risk_Level
Low       768
Medium    361
High      341
Name: count, dtype: int64
```

In [44]:
```
from sqlalchemy import create_engine

db_url = 'postgresql+psycopg2://postgres:123456@localhost:5432/ibm_hr_project'
engine = create_engine(db_url)
df.to_sql('ibm_hr_project', con=engine, if_exists='replace', index=False, chunks
print("Upload complete! Table 'ibm_hr_project' is ready in PostgreSQL.")
```

```
Upload complete! Table 'ibm_hr_project' is ready in PostgreSQL.
```

In [ ]:

In [ ]: